



Coordinación y Consistencia Distribuida

SEMANA 4 — TEMA 5

Unidad 1 — Generalidades de los Sistemas Distribuidos

Aplicaciones Distribuidas · Ingeniería de Software · UTEQ

Período 2026-2027 PPA · 25–31 mayo 2026

Prof. Guerrero Ulloa G.C.

Agenda de la Semana 4



Subtema 1

El problema de la consistencia — Teorema CAP

25 min



Subtema 2

Algoritmos de coordinación — Bully y Ring

30 min



Subtema 3

Transacciones distribuidas — Commit en dos fases (2PC)

30 min



Subtema 4

Sistemas de consenso — Paxos y Raft (introducción)

20 min

RA: Analiza las características y principios de los SD, evaluando ventajas, limitaciones y retos.

¿Sabían que...?

En 2017, Amazon S3 sufrió una caída de 4 horas que afectó a miles de sitios web y servicios en Internet. La causa raíz fue un problema de coordinación distribuida: un comando de mantenimiento eliminó más servidores de los planeados, y el sistema no pudo re-elegir líderes lo suficientemente rápido.

Fuente: Amazon Web Services Post-Mortem, marzo 2017

La coordinación distribuida no es un problema teórico — tiene impacto real y millonario.

Subtema 1: El Teorema CAP

El problema fundamental de la consistencia



Consistency

Todos los nodos ven los mismos
datos al mismo instante
(linealización)



Availability

Toda petición recibe una respuesta
(no necesariamente la más reciente)



Partition Tolerance

El sistema funciona ante la pérdida
arbitraria de mensajes entre nodos

Teorema de Brewer (2000): En un sistema distribuido, es imposible garantizar simultáneamente C, A y P.

El Triángulo CAP: Sistemas Reales

CP — Consistencia + Partición

HBase, ZooKeeper, etcd, MongoDB

Priorizan que los datos estén siempre correctos.

Si hay partición, rechazan lecturas/escrituras en nodos que no pueden verificar consistencia.

AP — Disponibilidad + Partición

Cassandra, DynamoDB, CouchDB

Priorizan que siempre haya respuesta.

Si hay partición, responden con datos que podrían estar desactualizados (stale reads).

CA (Consistencia + Disponibilidad) — Solo posible sin particiones de red (red 100% confiable). Ej: PostgreSQL standalone.

Ejemplo: Consistencia Eventual en DynamoDB (Amazon)

1. Escritura confirma en réplica principal (us-east-1) en ~5ms.
2. Réplicas secundarias reciben la actualización en 100–500ms.
3. Durante esa ventana → lectura obsoleta (stale read) posible.
4. Eventualmente todos los nodos convergen al mismo estado.

Subtema 2: Algoritmos de Coordinación

Elección de líder: Algoritmo Bully

¿Por qué necesitamos un líder?

En muchos SD, un nodo actúa como coordinador: gestiona locks, ordena escrituras, ejecuta protocolos como 2PC.

Si el coordinador falla, el sistema necesita elegir uno nuevo de forma automática y sin intervención humana.

El Algoritmo Bully (García-Molina, 1982) resuelve esto: el proceso con el ID más alto entre los activos se convierte en líder.

Pasos del Algoritmo Bully

1. P detecta falla del líder (timeout en heartbeat).
2. P envía ELECTION a todos los procesos con ID mayor.
3. Si alguien responde OK → P se detiene (alguien más grande tomará el control).
4. Si nadie responde → P se proclama líder y envía COORDINATOR a todos.
5. Quien recibe COORDINATOR actualiza su referencia al nuevo líder.

Elección de líder: Algoritmo Ring

Algoritmo de Elección en Anillo

Los procesos se organizan en un anillo lógico. Cada uno conoce a su sucesor.

Pasos:

1. P detecta falla del líder.
2. P crea mensaje ELECTION con su ID y lo envía a su sucesor.
3. Cada nodo agrega su ID al mensaje y lo reenvía.
4. Cuando el mensaje vuelve al iniciador, tiene todos los IDs activos.
5. Se elige al proceso con ID mayor.
6. Se envía mensaje COORDINATOR por el anillo.

Bully vs Ring — Comparación

Criterio	Bully	Ring
Topología	Cualquiera	Anillo lógico
Mensajes	$O(n^2)$ peor caso	$O(2n)$ siempre
Velocidad	Más rápido	Más lento
Robustez	Tolera caídas durante elección	Si un nodo cae durante paso, se atasca
Uso típico	Clústeres pequeños	Redes estructuradas

Subtema 3: Transacciones Distribuidas

Commit en dos fases (2PC) — Two-Phase Commit

Problema: Una transferencia bancaria debe restar de cuenta A (nodo 1) y sumar en cuenta B (nodo 2). Si el nodo 2 falla después de restar de A pero antes de sumar en B, ¿el dinero desaparece?

FASE 1 — PREPARE (Votación)

1. Coordinador envía PREPARE a todos los participantes.
2. Cada participante prepara la operación (escribe en WAL) sin confirmar.
3. Cada participante responde VOTE-COMMIT o VOTE-ABORT.

FASE 2 — COMMIT/ABORT (Decisión)

Si TODOS votaron COMMIT:

→ Coordinador envía GLOBAL-COMMIT.

Si ALGUNO votó ABORT:

→ Coordinador envía GLOBAL-ABORT.

→ Participantes deshacen los cambios (rollback).

Garantía: O TODOS confirman la transacción, o NINGUNO lo hace → Atomicidad distribuida.

Limitaciones de 2PC y la necesidad de consenso

Problemas de 2PC

Protocolo bloqueante: si el coordinador falla entre Fase 1 y Fase 2, los participantes quedan 'en limbo'.

Punto único de falla: todo depende del coordinador.

Latencia: requiere 2 rondas de comunicación síncronas con TODOS los participantes.

3PC — Intento de solución

Agrega una fase intermedia PRE-COMMIT para reducir el bloqueo.

Fase 1: PREPARE → votos.

Fase 2: PRE-COMMIT → aviso de intención.

Fase 3: DO-COMMIT → confirmación final.

Problema: 3PC no funciona bien con particiones de red.
Necesitamos algo mejor → Paxos y Raft.

¿Cuál es la solución moderna?

Los algoritmos de consenso (Paxos, Raft) resuelven el problema de ponerse de acuerdo en un valor ante fallos, SIN un coordinador único. Un quórum (mayoría) de nodos decide, lo que tolera que una minoría de nodos falle. Esto es lo que usan sistemas reales como etcd (Kubernetes), ZooKeeper (Kafka), y CockroachDB.

Subtema 4: Sistemas de Consenso

Paxos y Raft — Introducción

Paxos (Lamport, 1998)

Roles: Proposer, Acceptor, Learner.

Fases:

1. Prepare: Proposer envía número de propuesta.
2. Promise: Acceptors prometen no aceptar propuestas menores.
3. Accept: Si mayoría prometió, Proposer envía valor.
4. Accepted: Acceptors confirman.

Garantía: Si una mayoría acepta un valor, ese valor es la decisión consensuada. Tolerante a fallos (f fallos con $2f+1$ nodos).

Raft (Ongaro & Ousterhout, 2014)

Diseñado para ser entendible.

Estados: Follower, Candidate, Leader.

Mecanismos:

1. Elección de líder por términos (terms).
2. Replicación de log: líder replica entradas a seguidores.
3. Seguridad: solo líder con log más completo puede ganar elección.

Usado en: etcd (Kubernetes), CockroachDB, Consul, TiKV.

Recurso recomendado: <https://raft.github.io> — Visualización interactiva del algoritmo Raft

Resumen: Mapa de Coordinación Distribuida

Coordinación Distribuida

Consistencia (CAP)

CP vs AP
Eventual vs Fuerte

Elección de Líder

Bully $O(n^2)$
Ring $O(2n)$

Transacciones (2PC/3PC)

Atomicidad
Bloqueante

Consenso (Paxos/Raft)

Mayoría = f fallos
No bloqueante

Conclusión clave: En redes reales P es inevitable → elegimos entre C y A → necesitamos algoritmos de coordinación y consenso para mantener el sistema funcionando correctamente ante fallos.

Bibliografía

- [1] Brewer, E. (2012). CAP twelve years later: How the 'rules' have changed. IEEE Computer, 45(2), 23-29.
- [2] van Steen, M. & Tanenbaum, A. S. (2023). Distributed Systems, 4th ed. Maarten van Steen.
- [3] Kleppmann, M. (2017). Designing Data-Intensive Applications. O'Reilly Media.
- [4] Ongaro, D. & Ousterhout, J. (2014). In Search of an Understandable Consensus Algorithm. USENIX ATC.
- [5] Lamport, L. (1998). The Part-Time Parliament. ACM TOCS, 16(2), 133-169.
- [6] Burns, B. (2025). Designing Distributed Systems, 2nd ed. O'Reilly Media.